TECHNICAL REPORT · TP3D-2026-01 · 18 October 2026

TensorPool 3D Tier Partitioning

RTL Hierarchy Analysis and Implementation Assessment

Design target: TensorPool (`config=tensorpool`), MemPool repository, branch `yiczhang/3d-interco-tier`
Physical-design target: Synopsys 3DIC Compiler and Fusion Compiler, TSMC N7, two-tier face-to-face stack

Abstract

This report reconstructs the TensorPool RTL hierarchy from the source, quantifies every candidate tier-crossing interface, and evaluates four two-tier partitioning strategies. The recommended partition has been implemented: the RTL split exists on branch `yiczhang/3d-interco-tier` and both dies are floorplanned, power-planned and bonded in TSMC N7, so the figures below are measured rather than estimated.

The partition relocates the inter-Group TCDM interconnect — twelve 16x16 crossbars instantiated **inside** `mempool_group` rather than at cluster level, contrary to the module names — to a second tier. It costs **120,752** vertical connections, of which 99,072 are TCDM and the remainder AXI, DMA and configuration traffic that must cross because the compute die has no package I/O of its own. That figure is within 0.03% of the Group-on-Group fold, so the two partitions are indistinguishable in interface size and the case rests on floorplan: with the inter-Group network gone the Groups abut, and the compute die falls to **16.17 mm²** against **26.50 mm²** for the 2D cluster. Remote-Group latency drops from 9 cycles to **5** with an unregistered Group boundary, eliminating 147,840 flip-flops that existed only to break the long 2D wires.

Two constraints bound the result. Bonding is not free: the N7 AP layer sets a **3.60 µm** pitch floor, 9 µm microbump assembly does not fit, and the implemented array runs at 4.50 µm and 30.6% utilisation. And the interconnect die must match the compute die footprint for bonding while holding 0.143 mm² of logic — **0.89% utilisation**, and 22% more total silicon than the 2D part. Relocating L2 to that die is therefore a condition of the partition paying for itself, and it also relieves the package interface, which at 20,557 signals and 97% AXI is the binding constraint on the stack.

Contents

1 Scope and Methodology . . . 3
1.1 Objective . . . 3
1.2 Method . . . 3
1.3 Configuration under analysis . . . 3
2 TensorPool RTL Hierarchy and Communication Structure . . . 4
2.1 Where the crossbars actually are . . . 4
2.2 Hierarchy tree . . . 4
2.3 Interconnect inventory . . . 5
2.4 Interface widths . . . 5
2.5 Pipelining on the inter-Group path . . . 5
2.6 AXI, DMA and L2 path . . . 6
3 Candidate Tier Partitions . . . 6

3.1 Tier-crossing signal counts . . . 6
3.2 Option A — SubGroup-level folding . . . 7
3.3 Option B — Group-level folding . . . 8
3.4 Option C — Interconnect-centric partition . . . 8
3.5 Option D — Memory plus interconnect partition . . . 8
3.6 Comparison . . . 8
4 Recommended Partition . . . 9
4.1 Bond density feasibility . . . 9
4.2 Prior art in the repository . . . 10
5 Memory on the Top Tier Versus Interconnect on the Top Tier . . . 10
5.1 The case against relocating L1 . . . 10
5.2 Where a memory partition could work . . . 10
5.3 Filling the top tier without moving L1 . . . 11
5.4 Thermal considerations . . . 11
6 Required RTL Restructuring . . . 12
6.1 Option C . . . 12
6.2 Option C+ (additional) . . . 13
6.3 Options A and D (additional) . . . 13
6.4 Reducing the pipeline depth . . . 13
7 Synopsys 3DIC Compiler Capability Assessment . . . 13
7.1 Division of responsibility . . . 15
7.2 Suggested sequencing . . . 15
8 Implemented Design . . . 15
8.1 Measured figures . . . 15
9 Risks and Open Items . . . 16
10 Appendix: Source References . . . 18
10.1 RTL . . . 18
10.2 Synopsys documentation . . . 19
10.3 Reproduction . . . 19

1 Scope and Methodology

1.1 Objective

The question addressed is where to place the cut in a two-tier 3D implementation of TensorPool. The starting hypothesis, set by the architecture team, is that the second tier should not be used to fold the existing 2D floorplan (Groups stacked on Groups), but to relocate a level of the interconnect hierarchy — moving the complete logic, including routers, arbitration, registers and pipeline stages, not merely the wires.

The analysis covers four partitioning strategies, evaluates each against the actual RTL, and determines which parts of the flow can be handled by Synopsys 3DIC Compiler and which must be made explicit in the source.

1.2 Method

All structural statements in this report are derived from the RTL in `hardware/src/`. All parameter values and signal widths were obtained by elaborating `mempool_pkg` for `config=tensorpool`; none of the widths quoted here are hand-derived. All statements about Synopsys tool capability were verified against the local documentation set and the command reference, and no command is cited on the basis of its name alone.

Physical figures come from the implemented flow: port counts from the elaborated tier modules, die and Group dimensions from the floorplan, bond pitch and array utilisation from the technology file and the bond plan, and cell area from the placed interconnect die. Where a quantity has been measured, the measurement is quoted and its source named.

Source	Version / location
RTL	`hardware/src/`, branch `yiczhang/3d-interco-tier`
Configuration	`config/tensorpool.mk`, `config/config.mk`
Elaboration	QuestaSim 10.7b_1, Bender 0.32.1
Implemented design	`terapool-n7/tsmc7/3dic/`, flow `tensorpool_3d_v2_coi`
Technology	TSMC N7, 15-metal stack M0–M15 plus AP
Measured interface	`00e_d2d_interface.rpt`, `00_d2d_budget.rpt`
Measured area	`report_utilization` on the placed interconnect die
3DIC Compiler User Guide	X-2025.06 and Y-2026.03, 258 / 269 pp.
3DIC Compiler Tool Commands	Y-2026.03, 5361 pp.
3DIC Compiler brochure	November 2025 revision

1.3 Configuration under analysis

TensorPool selects the three-level hierarchy. `hardware/Makefile` adds `-DTERAPOOL` whenever `num_sub_groups_per_group != 1`, so TensorPool takes the `` `ifdef TERAPOOL `` branch of `mempool_cluster.sv` and `mempool_group.sv`, which introduces the SubGroup level between Group and Tile. The MemPool and MinPool flavours take the two-level branch and have a different hierarchy, different widths and different interconnect counts; results in this report do not transfer to them.

Parameter	Value	Parameter	Value
Cores	256	Banks per tile	32
Groups	4	Total L1 banks	2048
SubGroups per Group	4	Bank size	2 KiB
SubGroups	16	Total L1 (TCDM)	4 MiB
Tiles per SubGroup	4	Banking factor	8
Tiles	64	Superbanks per tile	2
Cores per tile	4	RedMulE instances	16 (8x32)

Parameter	Value	Parameter	Value
AXI masters per SubGroup	1	RedMulE TCDM ports	16 x 32 bit
DMA backends per Group	4	Remote-Group latency	**5 cycles**
L2 size / banks	4 MiB / 4	AXI data width	512 bit

2 TensorPool RTL Hierarchy and Communication Structure

2.1 Where the crossbars actually are

The single most consequential structural fact for 3D partitioning is that the arbitration logic of the TCDM network is not located where the module names suggest.

- **The cluster top level contains no crossbar.** In the `TERAPOOL` branch, `mempool_cluster.sv` contains only the DMA distribution mid-ends, AXI cuts, 192 sets of pipeline registers (lines 181–244) and a block of pure XOR-permutation `assign` statements (lines 395–408). What is informally called "the top-level interconnect" is physically a bundle of wires spanning the die, plus a register rank.
- **The inter-Group crossbar sits inside the source Group.** `mempool_group.sv:431–530` instantiates, for each remote Group `r`, one `burst_variable_latency_interconnect` configured as a fully connected crossbar with `NumIn = NumOut = 16`. With four Groups this gives **twelve 16x16 crossbars** in total. A request is routed to the destination Tile of the remote Group already on the sending side; the receiving Group performs no further arbitration and injects the request directly into the target Tile's local crossbar.
- **The inter-SubGroup network at Group level is also only wiring.** `mempool_group.sv:413–425` is a second XOR-permutation block. The actual SubGroup-to-SubGroup crossbars are inside `mempool_sub_group.sv:346–443`: sixteen SubGroups x three remote ports = **48 crossbars of 4x4**.
- **Tile-level arbitration uses `stream_xbar`**, `mempool_tile.sv:855` and `:877`, 27 inputs to 32 banks for requests and 32 to 27 for responses. The 27 initiators are 20 local ports (four cores plus sixteen RedMulE ports) and seven remote ports.

> **CONSEQUENCE**
> "Moving the top-level interconnect to the top tier" means, in RTL terms, relocating the `gen_remote_interco` generate block of `mempool_group` together with the cluster-level pipeline registers. `mempool_cluster` itself is already an empty shell and does not need to be moved.

2.2 Hierarchy tree

```
mempool_system                                          mempool_system.sv
+- mempool_cluster                                      mempool_cluster.sv   <- cluster top
|  +- idma_split_midend + idma_distributed_midend       :87-126
|  +- 192 x 4 pipeline register sets  (~145k FF)       :181-244   inter-Group pipeline
|  +- XOR permutation assigns (no logic)                :395-408   "top-level interconnect"
|  +- 4 x mempool_group                                 :273-389
|      +- 48 x 2 spill_register  (~24k FF per Group)    group.sv:202-231   latency = 9
|      +- 3 x burst_variable_latency_interconnect       group.sv:431-530
|      |        LIC 16x16   <- inter-GROUP crossbar
|      +- XOR permutation assigns (inter-SubGroup)      group.sv:413-425
|      +- idma_distributed_midend + 4 x axi_cut         group.sv:534-597
|      +- 4 x mempool_sub_group                         group.sv:273-407
|          +- 4 x mempool_tile                          sub_group.sv:151-238
|          |   +- 4 x mempool_cc (Snitch + FPU + IPU)   tile.sv:197-283
|          |   +- 1 x redmule_top 8x32 (tile 0 only)    tile.sv:1081-1124
|          |   +- snitch_icache                         tile.sv:286-330
|          |   +- stream_xbar 27->32 / 32->27           tile.sv:855 / :877
|          |   +- 2 x tcdm_wide_narrow_mux (superbank)  tile.sv:443-478
|          |   +- 32 x [tcdm_adapter + tc_sram 512x32]  tile.sv:635-687   L1 banks
|          +- 1 x LIC 4x4  (SubGroup local)             sub_group.sv:298-313
```

```
|          +- 3 x LIC 4x4  <- inter-SUBGROUP crossbar   sub_group.sv:346-443
|          +- axi_hier_interco + snitch_read_only_cache sub_group.sv:478   8 KiB RO$
|          +- 1 x axi_dma_backend + axi_xbar            sub_group.sv:532
+- L2: 4 banks (4 MiB) + axi_xbar + ctrl_registers + bootrom
```

2.3 Interconnect inventory

Level	Module and line	Count	Configuration	Wire length
Tile local	`mempool_tile.sv:855` / `:877`	64 x 2	`stream_xbar` 27⟶32 / 32⟶27	intra-tile
SubGroup local	`mempool_sub_group.sv:298`	16	LIC 4x4	intra-SubGroup
Inter-SubGroup	`mempool_sub_group.sv:346`	48	LIC 4x4, 1 spill stage	intra-Group
Inter-Group	`mempool_group.sv:431`	12	LIC **16x16**, 1 spill stage	**full die span**
Cluster	`mempool_cluster.sv:395`	0	XOR permutation only	**full die span**

2.4 Interface widths

All widths below were read out of an elaborated `mempool_pkg` under `config=tensorpool`.

Type	Bits	Composition
`tcdm_master_req_t`	108	payload 47 + wen 1 + be 4 + `tcdm_addr` 18 + burst 38
`tcdm_slave_req_t`	108	payload 47 + wen 1 + be 4 + `tile_addr` 14 + `tile_id` 4 + burst 38
`tcdm_master_resp_t`	144	payload 47 + grouped burst response 97
`tcdm_slave_resp_t`	148	payload 47 + `tile_id` 4 + grouped burst response 97
`tcdm_payload_t`	47	`meta_id` 6 + `core_id` 5 + amo 4 + data 32
`burst_t` / `burst_gresp_t`	38 / 97	`BurstLen` 16, request grouping 2, response grouping 4
`axi_tile_req_t` / `_resp_t`	717 / 528	AXI data width 512
`tcdm_dma_req_t` / `_resp_t`	606 / 527	wide DMA port, 16 words
`dma_req_t`	113	iDMA burst descriptor
`ro_cache_ctrl_t`	258	enable, flush, 4 address ranges

The request path is **width-preserving across the inter-Group crossbar**: 108 bits enter as `tcdm_master_req_t` and 108 bits leave as `tcdm_slave_req_t`, because the 18-bit TCDM address is replaced by a 14-bit tile address plus a 4-bit tile identifier. On the response path 148 bits enter and 144 bits leave. This property is what makes the crossbar boundary essentially free as a tier cut, and is used in Section 4.

2.5 Pipelining on the inter-Group path

In the 2D design two register banks sat on the inter-Group path, both present for no reason other than to break the long wires:

- `mempool_cluster.sv:181–244`: for each of the 4 x 3 x 4 x 4 = 192 channels, one `spill_register` on the master request, one `fall_through_register` on the master response, one `fall_through_register` on the slave request and one `spill_register` on the slave response — (2·108+2) + (144+1) + (108+1) + (2·148+2) = 770 flip-flops per channel, **147,840** in total.
- `mempool_group.sv:202–231`, at `RemoteGroupLatencyCycle == 9`: 48 channels per Group, two `spill_register` instances each, **97,536** flip-flops across four Groups.

`RemoteGroupLatencyCycle` selects how many of these ranks are instantiated (`mempool_pkg.sv:249–260`):

Value	Register ranks between a SubGroup and the Group port	Flip-flops
5	**none — the SubGroup drives the Group port directly**	**0**
7	the port rank	147,840

Value	Register ranks between a SubGroup and the Group port	Flip-flops
9	the port rank and the port-to-SubGroup rank	245,376
11	all three	343,000

`config/tensorpool.mk` sets **5**, and the source comment states why: "5 exists for the 3D stack, where the Group boundary is a bond rather than a traverse of the die and does not need a pipeline stage to be fast." The 192 cluster-level register sets were relocated into `mempool_group.sv:295–356` and made conditional on `RemoteGroupLatencyCycle >= 7`, so in this configuration they are not instantiated at all; `mempool_cluster.sv` is 209 lines holding two die instances and nothing else.

The partition therefore does not merely avoid adding a pipeline stage — it removes 147,840 flip-flops and four cycles of remote-access latency that the 2D floorplan required. A 4.5 µm pitch array directly over the Group footprint replaces a traverse of a 5.3 mm die.

> **WHY THE REGISTERS COULD GO**
> The cluster-level registers were not purely a timing measure: the source comment states that they "break dependencies between request and response, establishing a correct valid/ready handshake". That function did not have to be re-engineered, because it was never only in those registers. `burst_variable_latency_interconnect` carries `SpillRegisterReq`, `SpillRegisterResp` and `FallThroughRegister`, all enabled (`tensorpool_inter_group_interco.sv:145–149`). Relocating the crossbar moved that buffering across the bond with it. `matmul_i32` and `dotp_i32` pass with 0 errors at 5, 7, 9 and 11 cycles.

2.6 AXI, DMA and L2 path

A second, largely independent network carries instruction fetch, L2 traffic and DMA. Most of its logic is at SubGroup level: each SubGroup instantiates one `axi_hier_interco` containing one 8 KiB `snitch_read_only_cache` (`mempool_sub_group.sv:478`), one `axi_dma_backend` with a 1-to-2 `axi_xbar` (`:532`), and exposes one AXI master port. Group level adds only four `axi_cut` instances and an iDMA distribution mid-end. The sixteen AXI master ports of the cluster connect in `mempool_system.sv` to a 4-bank, 4 MiB L2, an AXI crossbar, the control registers and the boot ROM. This path is latency-tolerant by construction and is relevant to the area-balance discussion in Section 5.3.

3 Candidate Tier Partitions

3.1 Tier-crossing signal counts

Each figure below counts payload bits plus valid and ready for every channel that crosses the tier boundary, using the widths of Section 2.4.

One property of this partition governs the whole count: **the compute die has no package I/O**. It is bonded face-to-face to the interconnect die and reaches the outside world only through it, so every signal that leaves a Group crosses the bond — AXI, DMA and configuration included, not only TCDM. The measured interface of the implemented compute die is **120,752** signals:

Bus family	Per Group	Total	Share
`tcdm_master_req` 108 + 2	5,280	21,120	17.5%
`tcdm_master_resp` 144 + 2	7,008	28,032	23.2%
`tcdm_slave_req` 108 + 2	5,280	21,120	17.5%
`tcdm_slave_resp` 148 + 2	7,200	28,800	23.8%
TCDM subtotal — 48 channels x 516 bits	**24,768**	**99,072**	**82.0%**

Bus family	Per Group	Total	Share
`axi_mst_req` 717 x 4 masters	2,868	11,472	9.5%
`axi_mst_resp` 528 x 4 masters	2,112	8,448	7.0%
DMA request, ready, metadata	117	468	0.4%
`wake_up` 64 cores	64	256	0.2%
`ro_cache_ctrl` 258	258	1,032	0.9%
Per Group	**30,187**	**120,748**	
Clock, reset, test mode, scan enable	—	4	
Total, measured		**120,752**	**100%**

Source: `00e_d2d_interface.rpt`, produced from the elaborated tier modules; the bond plan in `00_d2d_budget.rpt` agrees. Note the TCDM array shape, `[NumGroups-1:0][NumGroups-1:1][NumSubGroupsPerGroup-1:0][NumTilesPerSubGroup-1:0]`: the second index starts at 1, so only the three **remote** ports cross and the local one stays inside the Group.

The AXI share is the reason the L2 question of Section 5.3 is structural rather than a matter of filling area: a sixth of the bond array exists only to carry traffic to a memory that sits on the far side of the package boundary.

Set against the other candidate cuts:

Cut location	Option	Total	Timing character
Below the inter-Group crossbar (whole compute die, **measured**)	**C**	**120,752**	unregistered at 5 cycles
As C, plus below the inter-SubGroup crossbar	**C+**	219,000	one spill stage present
`mempool_group` ports (Group-on-Group fold)	B	120,788	same ports, same count
`tc_sram` pins	A / D	161,792	**combinational, 1 cycle**
Above `tcdm_adapter`	A / D	266,240	valid/ready, no pipeline
Complete tile memory subsystem	A	522,048	valid/ready, no pipeline
Cluster AXI ports (L2 to top tier)	—	19,920	AXI, fully pipelined

The Group-port figure is measured on the implemented `mempool_group`, which presents **30,197** ports (`01d_group_abstract.log`).

> **KEY RESULT**
> Options B and C cross **the same interface to within 0.03%** — 120,788 against 120,752. Once the crossbars have moved out of the Group, the compute die is four Groups and nothing else, so cutting below the crossbar and cutting at the Group boundary cut the same wires.
>
> Bond count therefore does not distinguish the two partitions. What distinguishes them is what each does to the floorplan, and that difference is large: see Section 4.1.

3.2 Option A — SubGroup-level folding

Partition each SubGroup across the two tiers, typically by placing the L1 memories on the top tier and keeping the cores on the bottom tier.

There is no RTL boundary to cut at. The banks are instantiated inline in the `gen_banks` generate block of `mempool_tile.sv:635–687`; no `mempool_tile_mem` module exists. More seriously, the `tcdm_adapter`-to-`tc_sram` path is a hard single-cycle loop: `out_req_o` is asserted and `out_rdata_i` must be valid in the following cycle. Crossing the tier boundary there adds latency to **every** L1 access, including the local ones that the MemPool memory model guarantees at one cycle. Vertical connection counts range from 161,792 to 522,048 depending on exactly where the cut is taken.

3.3 Option B — Group-level folding

Keep the SubGroups and the interconnect intact and fold whole Groups onto the second tier. This aligns perfectly with the existing RTL: `mempool_group` is already hardened as a macro in the 2D flow (see the `PostLayoutGr` hook at `mempool_pkg.sv:265` and `mempool_cluster.sv:275`), and the boundary already carries registers on both sides.

This is the option implemented on the existing `mbertuletti/3d` branch, where `hardware/src/terapool_top_die.sv` places Groups 2 and 3 on the upper die and Groups 0 and 1 on the lower die. It is a sound bring-up vehicle for the 3D flow, but it does not address the stated objective: the interconnect stays on the compute tier, the inter-Group wires still have to cross each die, and the routing channels between Groups are unchanged.

3.4 Option C — Interconnect-centric partition

Keep the compute-dominated SubGroups on the bottom tier and move the Group-level interconnect to the top tier. In RTL terms this relocates the twelve `burst_variable_latency_interconnect` instances of `mempool_group.sv:431–530`, each now wrapped as `tensorpool_inter_group_interco`.

The cost is **120,752** vertical connections (Section 3.1). No pipeline stage is added; four cycles are removed. The bottom tier retains only intra-Group routing, so the channels that separate the four Groups are eliminated outright — `COMPUTE_CHANNEL_FRAC` is 0 in the implemented floorplan — and the top tier provides a complete, otherwise unused metal stack for the inter-Group network.

This is the option that was built; Section 8 reports what it measures.

The **C+** variant additionally relocates the 48 inter-SubGroup 4x4 crossbars of `mempool_sub_group.sv:346–443`, bringing the total to approximately 219,000 vertical connections — 120,752 plus the 48 inter-SubGroup channel sets — and reducing each SubGroup to a purely local compute island. This variant is attractive for a second reason: the `tcdm_sg_*` ports of `mempool_sub_group.sv:41–53` already exist, making it the only cut in this study that requires no new module ports.

3.5 Option D — Memory plus interconnect partition

Move both the L1 memories and the higher-level interconnect to the top tier. This combines the benefit of C with the cost of A. Its motivation is area balance rather than routing: with only the interconnect on the top tier, that tier is sparsely utilised. Section 5 argues that the area-balance problem has cheaper solutions.

3.6 Comparison

Criterion	A	B	C / C+	D
Alignment with existing RTL hierarchy	none	exact	crossbar had to be extracted	none
Vertical connection count	162k–522k	121k	**121k / 219k**	260k+
Additional pipeline stages required	yes	no	**two removed**	yes
Compute-die area	—	26.5 mm²	**16.2 mm²**	—
Relief of bottom-tier routing congestion	negligible	none	**substantial**	substantial
Group placement density improvement	none	none	**yes**	yes
Headroom for bandwidth increase	none	none	**yes**	yes
Top-tier area utilisation	high	high	**0.89% as built**	high
Total silicon versus 2D	—	equal	**+22% unless filled**	—
Physical-design complexity	very high	low	moderate	very high
Impact on programming model	severe	none	**none**	severe

Two rows carry the decision. Vertical connection count does not separate B from C — both are the Group port list — so the choice is made on compute-die area, where C gains 39%. Against that, C leaves the second die at 0.89% utilisation, which makes filling it a precondition rather than a refinement.

---

4 Recommended Partition

The recommendation is **Option C as the first step, with C+ as a follow-on**, and the design has been built this way. Three measurements support it.

The compute die shrinks by 39%. This is the substance of the partition. In 2D the four Groups need a channel between them and each Group needs a channel between its SubGroup columns, sized at 0.18 and 0.55 of a Group and SubGroup pitch respectively; the cluster comes to **26.50 mm²**. With the inter-Group network gone the Groups abut — `COMPUTE_CHANNEL_FRAC` is 0 — and the Group channel falls to a fixed 100 µm, giving a Group of 2008.5 µm against 2432.3 µm and a compute die of **16.17 mm²**. Sixteen SubGroups abutted with no channel at all would be 14.51 mm², so the floorplan is within 11% of the floor set by the hard macros.

Four cycles of remote latency were removed. Section 2.5: `RemoteGroupLatencyCycle = 5` with an unregistered Group boundary, eliminating 147,840 flip-flops that existed only to break the 2D wires. The partition does not merely avoid adding a pipeline stage; it pays one back.

The cluster level is empty, so relocating the crossbars breaks no module. `mempool_cluster.sv` is 209 lines containing two die instances, five elaboration assertions and nothing else. The twelve crossbars sit in `tensorpool_interco_tier.sv:293–314` as `tensorpool_inter_group_interco`, each one `burst_variable_latency_interconnect` with `NumIn = NumOut = 16`.

C is preferred over C+ as the first step because C+ adds the 48 inter-SubGroup channel sets to a cut that already carries everything else, taking the interface to roughly 219,000 and the array from 30.6% to about 55% utilisation, for wires that stay inside a Group and are already short.

4.1 Bond density feasibility

The pad sits on AP. `AP.W.1` and `AP.S.1` are both 1.80 µm, so the pitch floor this technology supports is **3.60 µm**. Anything below that needs a dedicated Cu–Cu bonding level, which this PDK does not describe; sub-micrometre hybrid-bonding pitches quoted in the literature are therefore not numbers this process can be asked for.

The array is placed over each Group footprint, inset by one pitch, with half the sites reserved for power and ground. Sites per Group and the resulting utilisation, from `00_d2d_budget.rpt`:

Pitch	Pad	Sites per Group	Signal sites	Utilisation	Verdict
1.00 µm	0.50	2006 x 2006	2,012,018	1.5%	below the AP floor, needs a bonding level
2.00 µm	1.00	1002 x 1002	502,002	6.0%	below the AP floor, needs a bonding level
3.60 µm	1.80	555 x 555	154,012	19.6%	fits, at the PDK floor
4.50 µm	**1.80**	**444 x 444**	**98,568**	**30.6%**	**fits — implemented**
9.00 µm	1.80	221 x 221	24,420	123.6%	**does not fit**

Against the measured 30,187 signals per Group, the loosest pitch that still fits is **8.14 µm**. Microbump assembly at 9 µm is therefore out: the array is 24% short of the sites the interface needs. Hybrid bonding is required, and the window is 3.60 to 8.14 µm.

At the implemented 4.50 µm the array runs at 30.6% utilisation — a factor of 3.3 of headroom, which is what leaves room for the C+ variant and for a future bandwidth increase.

> **A RULE DEVIATION THAT NEEDS SIGN-OFF**
> The die-to-die via under the pad is sized from the pad rather than from the layer rule: `RV` minWidth is 2.70 µm against a 1.80 µm pad, so the cut is set to half the pad and `create_d2d_vias` warns. This is deliberate and recorded in the flow, but it is a departure from the foundry rule deck and must be agreed with the foundry before tape-out.

4.2 Prior art in the repository

The branch `origin/mbertuletti/3d` already contains related work and should be reviewed before any implementation starts:

Commit	Subject	Relevance
`043eab77`	Move cluster registers within the Group	Directly reusable; makes the Group a registered-IO macro
`90682bb9`	Add upper die (`terapool_upper_die.sv`)	Implements Option B, not Option C
`072b3fae`	Rename “upper_die” to “top_die”	Naming convention to follow

Commit `043eab77` moves the four cluster-level register sets into `mempool_group`, giving the Group fully registered TCDM interfaces at constant total latency. That refactor is a prerequisite for a clean Option C cut and should be adopted rather than reinvented. The remaining two commits implement the Group-on-Group fold and are orthogonal to the partition proposed here.

---

5 Memory on the Top Tier Versus Interconnect on the Top Tier

5.1 The case against relocating L1

Vertical connection cost is 1.6x to 5x higher. With 2048 banks, the cheapest cut — directly at the `tc_sram` pins — requires 161,792 connections; cutting above `tcdm_adapter` requires 266,240; relocating the complete tile memory subsystem requires 522,048.

The timing consequence is structural. The bank path is a hard single-cycle loop. Crossing tiers there adds latency to every L1 access, local accesses included. MemPool’s defining property is one-cycle local and at most five-cycle remote L1 access; this partition removes it.

There is no module boundary to cut at. The banks are inline in a generate block; a new `mempool_tile_mem` module would have to be created from nothing.

The routing benefit is close to zero. L1 consists of 2048 small 2 KiB macros tightly interleaved with the tile logic. The tile-internal crossbar wires are already short. Relocating the memories trades area without relieving any congestion.

5.2 Where a memory partition could work

Two qualifications are worth recording, because they bound the argument rather than overturn it.

First, a die-to-die hybrid bond is not inherently a pipeline stage. Its parasitic load is comparable to a few hundred micrometres of metal. If the memory tier were floorplanned directly above its own tile, with bonds landing inside the tile footprint, the crossing itself could be combinational; the problem is then the horizontal routing to and from the bond array, not the bond. At a 2 µm pitch a 250 x 250 µm tile offers roughly 15,000 bond sites against a requirement of 2,528, so the density is available.

Second, the TCDM interconnect is explicitly built for variable target latency — the module is named `burst_variable_latency_interconnect`, responses carry initiator metadata and are reordered by ID in

`tcdm_shim`. Adding latency to a **subset** of banks is therefore functionally supported. Moving one of the two superbanks per tile to the top tier would produce a non-uniform L1 with a fast and a slow half.

Both observations describe an **architectural** change, not a physical partition: they alter the memory model that software and the published performance results rely on. They should be evaluated as a separate study, not folded into the first 3D implementation.

5.3 Filling the top tier without moving L1

The area imbalance of Option C is the partition's principal cost, and it is severe.

The placed interconnect die holds **143,442 µm²** of standard cells and no macros, on a die of **16,166,431 µm² — 0.89% utilisation**. The die is that size not because its contents need it but because it must match the compute die footprint for the bond array to land.

The consequence is a total-silicon figure. The 2D cluster is 26.50 mm² the 3D part is 16.17 + 16.17 = **32.33 mm²**, which is **22% more silicon** in exchange for a compute die 39% smaller and an interconnect that no longer crosses it. Whether that trade is worth making depends entirely on what else goes onto the second die, so the first remedy below is a condition of the partition paying for itself rather than a refinement.

Three remedies, in order of attractiveness:

1. **Relocate the L2 subsystem.** L2 is 4 MiB across four banks — the same capacity as the entire L1 — and sits behind AXI with tens of cycles of latency, so it is completely latency-tolerant. Cutting at the sixteen AXI master ports of `mempool_cluster` costs only **19,920 vertical connections**. This is by a wide margin the cheapest way to add memory area to the top tier, and it also moves the AXI system crossbar, control registers and boot ROM off the compute tier.
2. **Execute C+ and relocate the SubGroup AXI subsystem.** The 48 inter-SubGroup crossbars, the sixteen `axi_hier_interco` instances with their 8 KiB read-only caches, the sixteen DMA backends and their AXI crossbars are all system logic whose relocation does not touch the TCDM latency contract.
3. **Accept an asymmetric stack.** The top tier does not have to match the bottom tier in area, and it can use an older, cheaper process — 3DIC Compiler supports per-die technologies and scale factors (User Guide, Chapter 4). For a research vehicle a smaller interconnect die is a legitimate answer.

The resulting recommended top-tier content is: inter-Group crossbars, L2 with its AXI infrastructure, and optionally the inter-SubGroup crossbars and the SubGroup AXI subsystem. The cluster pipeline registers are no longer on the list — at `RemoteGroupLatencyCycle = 5` they do not exist.

Relocating L2 is attractive for a second reason beyond area. The interconnect die is the only die bonded to the substrate, and its package-facing interface is **20,557** signals of which 97% is the sixteen AXI master ports — more than a C4 array on a 4.0 mm die can carry. Putting L2 on the interconnect die fills it and removes most of that AXI from the package boundary, turning two open problems into one solved one.

Revised vertical cost: **approximately 120,800** connections for C-plus-L2, since the AXI ports move from the package boundary to internal nets on the same die rather than adding to the bond count; and approximately **219,000** for C+-plus-L2.

5.4 Thermal considerations

Concentrating all compute logic on one tier is favourable for heat extraction: the power density is confined to a single plane rather than stacked, and in a face-to-face assembly the compute die can be placed on the side facing the heat sink, with the low-activity interconnect and L2 die on the far side. This is a significant secondary argument for Option C over any partition that splits compute across both tiers.

The assembly orientation must be verified rather than assumed, because it interacts with power delivery: whichever die is furthest from the C4 bumps needs TSVs for its power network. 3DIC Compiler provides `analyze_3d_thermal` with the built-in engine or RedHawk-SC Electrothermal (User Guide, Chapters 11–12) and `analyze_3d_rail` for EM/IR, and both can be run during feasibility exploration before any RTL is written.

6 Required RTL Restructuring

Changes are classified as **(1)** strictly required by the Synopsys flow, **(2)** not required but materially simplifying physical design, or **(3)** an architectural modification rather than a physical partition.

6.1 Option C

Change	Detail	Class
Two tier top-level modules	Built as `tensorpool_compute_tier` and `tensorpool_interco_tier`, each a complete synthesisable design. The 3D top-level netlist must instantiate dies that are already implemented separately (User Guide, Chapter 7).	1
3D top-level netlist	`tensorpool_3d_top`, containing only the two die instances and an optional empty interposer placeholder. This is the input to `create_3d_top_design` and `read_verilog`.	1
Every die-to-die signal must be a port	Bumps and bond pads attach to ports; internal nets cannot be assigned.	1
Per-die clock and reset ports	Today a single `clk_i` fans out to the whole cluster. Each die needs its own clock entry for independent CTS, and die-to-die skew must be constrained in multi-die STA.	1
Move cluster registers into the Group	Follows `043eab77`. Gives the Group registered TCDM interfaces and places one register rank on the bottom side of the cut at unchanged total latency.	2
Extract `gen_remote_interco` into a module	`mempool_group.sv:431–530` is an inline generate block using Group-internal signals. Extracting it into `tensorpool_inter_group_interco` is mechanical; without it the twelve instances can only be lifted by netlist-level `group` / `ungroup` operations, which is fragile.	2
Explicit die-to-die interface bundle	A named packed type plus a flat, sorted port naming scheme such as `d2d_g<g>_sg<sg>_t<t>_*`. `derive_3d_connections` connects die-internal nets to top-level ports **by name**, so a regular naming scheme makes bump planning fully scriptable and CSV-driven.	2
Per-die scan chains	`scan_data_i` / `scan_data_o` are currently left unconnected (for example `mempool_cluster.sv:280–281`). A manufacturable design needs per-die test access under IEEE 1838.	2
Change `RemoteGroupLatencyCycle`	Done, and further than proposed: the ladder now has a 5 that instantiates no rank at all, removing 147,840 flip-flops and four cycles relative to the original 9. Visible to software and to the published performance model.	3

Change	Detail	Class
Widen the inter-Group network	Changing `NumIn` / `NumOut`, the burst grouping factors `GROUP_REQ` / `GROUP_RSP`, or the number of remote ports per tile.	3

6.2 Option C+ (additional)

Extract the three 4x4 crossbars of `mempool_sub_group.sv:346–443` into a module and relocate them to the top tier. The `tcdm_sg_*` ports of `mempool_sub_group.sv:41–53` then become die-to-die ports directly; no new ports are needed. `mempool_group` degenerates into a pure container and could be removed as an RTL level, although keeping it is harmless and preserves the existing hardening flow.

6.3 Options A and D (additional)

Create a `mempool_tile_mem` module containing `gen_banks`, the `tcdm_wide_narrow_mux` instances and possibly both `stream_xbar` instances **(class 1)**; insert pipeline stages on the bank path and modify the `tcdm_adapter` handshake, which currently assumes the SRAM responds in the next cycle **(class 3)**; and update every latency assumption in `software/runtime` **(class 3)**.

6.4 Reducing the pipeline depth

A shorter physical interconnect permits a shorter pipeline, and the ladder of Section 2.5 exposes the range as a single configuration value. At 5 no rank is instantiated between a SubGroup and the Group port; `gen_tcdm_port_feedthrough` (`mempool_group.sv:357–375`) wires all four TCDM buses straight through and the Group boundary is combinational. This is the shipping configuration, and `matmul_i32` and `dotp_i32` pass with 0 errors at 5, 7, 9 and 11.

Whether 5 is the floor is open. The bond replaces a die traverse, so the limiting path is now inside the crossbar rather than between Groups — a question for post-route timing on the interconnect die, not for configuration.

---

7 Synopsys 3DIC Compiler Capability Assessment

3DIC Compiler is an assembly, analysis and verification platform, not an RTL partitioning tool. Its documented flow assumes that each die is an independently implemented design — produced in Fusion Compiler — which is then instantiated in a 3D top-level netlist, placed with `create_die` and `set_cell_location -z_offset`, connected through bumps and bond pads, checked and analysed. No command exists that splits one netlist into two dies.

Capability sought	Finding	Source
Die / tier partitioning	Not present. The term does not appear anywhere in the User Guide in the sense of die assignment.	User Guide, full text
Hierarchy assignment to dies	Not present. Every `*_3d_*` command concerns bumps, TSVs, stacking, routing, checking, thermal analysis or AI-driven exploration.	Command reference, full command list
Hierarchy push / pull	`push_down_objects` and `pop_up_objects` operate on **pseudo bump regions** in the physical hierarchy, not on logical hierarchy.	User Guide, Ch. 5

Capability sought	Finding	Source
`auto_partition_design`	Exists, but is the conventional 2D hierarchical-partitioning flow: it regroups **existing logical hierarchies** into virtual groups under cell-count and pin-count constraints. Output is hierarchy within one block, not two dies, and its quality is bounded by the RTL hierarchy it starts from.	Command reference, p. 256
`partition_block`	Clips a physical region into a standalone block. The manual states it "is always a valid design, though not always a complete sub-circuit" and "makes no additional edits to clean up incomplete states such as dangling nets, missing drivers". An analysis aid, not an implementation path.	Command reference, p. 2844
Blocks spanning dies	Not supported. Each die is a separate NDM block; the top level only instantiates and places them.	User Guide, Ch. 7
3D interface definition	Supported, at the physical level. `derive_3d_interface` derives bump, TSV and bond-pad objects from existing ones; `create_3d_virtual_blocks` creates virtual interface blocks for early exploration; `derive_3d_connections` connects die-internal nets to top-level ports **by name**.	User Guide, Ch. 6
Vertical interconnect assignment	Strong. `create_bump_region`, `create_3d_mirror_bumps`, `assign_3d_interchip_nets` (shortest-wirelength assignment), `propagate_3d_connections`, `commit_pseudo_bumps`; locations and connections can be driven from CSV via `read_design_io` and `read_block_connection_file`	User Guide, Ch. 5 and 7

Capability sought	Finding	Source
Cross-die optimisation	Not logical optimisation. 3DSO.ai searches configuration space (bump density, placement, thermal parameters); `analyze_3d_thermal` and `analyze_3d_rail` cover multiphysics; cross-die timing is StarRC plus PrimeTime hyperscale. There is no cross-die synthesis or retiming.	Brochure; User Guide Ch. 11–14

7.1 Division of responsibility

Must be expressed in RTL

- Split into two independently implementable die top modules
- Die-to-die signals as ports on both modules
- The 3D top-level netlist
- Per-die clock, reset and scan
- Die-to-die port naming convention
- Any change in pipeline depth

Handled by 3DIC Compiler

- Bump, bond-pad and TSV array creation and planning
- Bump mirroring between dies and legality checking
- Shortest-wirelength signal-to-bump assignment
- Die placement, z-ordering and overlap checking
- More than 60 classes of 3D design rule check
- Thermal, EM/IR and SI analysis; feasibility exploration
- Per-die technologies and scale factors

7.2 Suggested sequencing

Feasibility analysis does not require a PDK and does not require any RTL change. Before committing to the restructuring of Section 6, `create_3d_virtual_blocks` and the feasibility flow (User Guide, Chapter 12) can be used to establish the bond area required for 99,000 to 197,000 connections at the candidate pitch, and to obtain first thermal and IR estimates. Only after that should the RTL work begin.

8 Implemented Design

`mempool_cluster` instantiates two dies, `tensorpool_compute_tier` and `tensorpool_interco_tier`. Both have been floorplanned, power-planned and bonded in TSMC N7 under `terapool-n7/tsmc7/3dic/`; the functional equivalence of the split is recorded in `doc/3d/VERIFICATION.md`, which reports identical cycle counts across `matmul_i32`, `dotp_i32`, `synth_i32` and the test suite.

8.1 Measured figures

Quantity	Value	Source
Compute-die vertical interface	120,752	`00e_d2d_interface.rpt`
— TCDM	99,072	
— AXI	19,920	
— DMA, wake-up, cache configuration	1,756	
— clock, reset, test, scan enable	4	
Group ports	30,197	`01d_group_abstract.log`

Quantity	Value	Source
Group footprint	2008.5 µm square	floorplan
Compute die	4020.8 µm square, 16.17 mm²	floorplan
Interconnect die	4020.8 µm square, 16.17 mm²	floorplan
Interconnect-die cell area	0.143 mm², 0.89%	`report_utilization`
2D cluster, same formulas	26.50 mm²	`00_d2d_budget.rpt`
Bond pitch floor, AP layer	3.60 µm	`AP.W.1`, `AP.S.1`
Bond pitch, implemented	4.50 µm	bond plan
Bond array utilisation	30.6%	`00_d2d_budget.rpt`
Package-facing interface	20,557, 97% AXI	`00_d2d_budget.rpt`
Remote-Group latency	5 cycles	`config/tensorpool.mk`

9 Risks and Open Items

Item	Status
Area split by synthesis	The interconnect die is placed and measures 0.143 mm² of cells. The compute die is macro-dominated and its Group content has not been synthesised in this flow, so its area is a floorplan figure rather than a synthesis one.
Package interface	**The binding constraint.** 20,557 signals, 97% of them the sixteen AXI master ports, on the one die bonded to the substrate. This does not fit a C4 array on a 4.0 mm die. The cluster is a block inside an SoC rather than a standalone package, so the resolution is either to put L2 on the interconnect die or to narrow the off-stack AXI.
Top-tier content	**A precondition, not a refinement.** At 0.89% utilisation the partition costs 22% more silicon than the 2D part. Relocating L2 is the first remedy and also relieves the package interface above.
Power delivery	The die furthest from the C4 bumps requires a TSV-based power network. The bond budget reserves 50% of sites for power and ground and reports 398,416 PG terminals at 13.5 µA each, which is not the constraint; the interconnect die carrying both dies' current is.
Signoff timing	`RemoteGroupLatencyCycle = 5` simulates correctly at 5, 7, 9 and 11, but neither die has been through signoff timing. The 5-cycle configuration is not yet shown to close.
AXI latency	`mempool_group.sv:546–564` places four `axi_cut` per Group below the boundary and `tensorpool_interco_tier.sv:244–261` sixteen above it, so the crossing sits between two register ranks. Group-to-L2 latency grew by one cycle; the TCDM path is unaffected.
Per-die scan	`tensorpool_interco_tier.sv:41–42` declares `scan_data_i` and `scan_data_o`; neither is used, `scan_data_o` is undriven, and the compute tier has no scan data port. Listed as a class-2 change in Section 6 and unimplemented.
Metal stack hand-off	The SubGroup frame blocks to M9, the 2D SubGroup routes to M11, and the Group picks up vertical ports on M11. To be reconciled before assembly.

Item	Status
Bond rule deviation	The die-to-die via cut is sized from the 1.80 µm pad against an `RV` minWidth of 2.70 µm. Needs foundry agreement.
2D reference QoR	The published 2D floorplan results should be re-measured on the same RTL revision to give a clean baseline for the 3D comparison. The 26.50 mm² used here is computed by the 3D flow with the 2D channel fractions, not taken from a 2D run.
Interaction with `mbertuletti/3d`	That branch implements Option B and carries a large number of unrelated changes (RedMulE parametrisation, tile merge, address slicer, software). Only commit `043eab77` should be adopted directly.
Non-TERAPOOL configurations	MemPool and MinPool use the two-level branch, which contains its own `gen_remote_interco` at `mempool_group.sv:829`. Any refactor must either preserve that path unchanged or apply the same treatment consistently.

10 Appendix: Source References

10.1 RTL

Statement	Reference
TensorPool selects the three-level `TERAPOOL` hierarchy	`hardware/Makefile:106-108` ; `config/tensorpool.mk`
Cluster top level has no crossbar	`mempool_cluster.sv:181-244` , `:395-408`
Inter-Group crossbar: 12 x LIC 16x16	`mempool_group.sv:431-530` (instantiation at `:486`)
Inter-SubGroup connection at Group level is wiring only	`mempool_group.sv:413-425`
Inter-SubGroup crossbar: 48 x LIC 4x4	`mempool_sub_group.sv:346-443` (instantiation at `:399`)
`tcdm_sg_*` ports already exist	`mempool_sub_group.sv:41-53`
Inter-Group pipeline, approximately 242k flip-flops	`mempool_cluster.sv:181-244` ; `mempool_group.sv:202-231`
Remote-Group latency raised from 7 to 9	`config/tensorpool.mk` versus `config/terapool.mk`
L1 banks are inline in the tile, no module boundary	`mempool_tile.sv:635-687`
SRAM path is a hard single-cycle loop	`mempool_tile.sv:650-687`
Tile crossbars 27→32 and 32→27	`mempool_tile.sv:855` , `:877`
RedMulE on tile 0 of each SubGroup, 16 ports	`mempool_tile.sv:1081-1152` ; `mempool_pkg.sv:283-293`
SubGroup and Group are already hardened as macros	`mempool_pkg.sv:264-265` ; `mempool_cluster.sv:275` ; `mempool_group.sv:291`
SubGroup AXI subsystem and read-only cache	`mempool_sub_group.sv:478` , `:532`
L2, AXI crossbar, control registers, boot ROM	`mempool_system.sv:136` onwards
All parameter and width values	QuestaSim elaboration of `mempool_pkg` , `config=tensorpool`

10.2 Synopsys documentation

Statement	Reference
3D flow requires independently implemented dies and a 3D top-level netlist	User Guide Ch. 3 (Fig. 9, 10), Ch. 5, Ch. 7
No die-partitioning command exists	User Guide, full text; Command reference, full command list
`auto_partition_design` is 2D hierarchical partitioning	Command reference, p. 256
`partition_block` clips a physical region and does not guarantee a complete sub-circuit	Command reference, p. 2844
`push_down_objects` / `pop_up_objects` apply to pseudo bump regions	User Guide Ch. 5, “Bump Planning Flow”
Bump, TSV, mirroring, assignment and checking commands	User Guide Ch. 5, 6, 7
Thermal, feasibility, EM/IR analysis	User Guide Ch. 11, 12, 13
Hybrid bonding, face-to-face and face-to-back, direct 3D stacking, heterogeneous processes	Brochure; User Guide Ch. 4
Multi-die STA and DFT	Brochure (StarRC and PrimeTime hyperscale; IEEE 1838, TestMAX Manager)

10.3 Reproduction

The parameter and width values in Sections 2.3 and 2.4 can be reproduced by generating the Bender compile script with the full `config=tensorpool` macro set, compiling it in QuestaSim, and elaborating a module that imports `mempool_pkg` and prints `$bits` of each type. The tier-crossing counts of Section 3.1 follow from those widths by summing payload plus valid plus ready over the channel counts given in Sections 2.1 and 2.3.

End of report. No RTL was modified in the preparation of this analysis.